MÓDULO 1 — FUNDAMENTOS DE PYTHON

Introdução à Programação com Python

Uma linguagem simples, poderosa e perfeita para quem está começando no mundo do desenvolvimento e da análise de dados.

Aula prática com Google Colab

Por que aprender Python?

A LINGUAGEM IDEAL PARA INICIANTES

Python é uma das linguagens de programação mais populares do mundo, e por boas razões.

> Python é a linguagem que mais cresce no mundo e a mais recomendada para quem quer entrar na área de tecnologia.

Sintaxe simples e legível

O código se parece com inglês, facilitando muito o aprendizado inicial.

Altamente Versátil

Usada em automação, ciência de dados, desenvolvimento web e inteligência artificial.

Grande comunidade

Milhões de tutoriais, fóruns e bibliotecas disponíveis gratuitamente na internet.

Alta demanda no mercado

Uma das habilidades mais requisitadas por empresas de tecnologia atualmente.

Resultados rápidos

Em poucas linhas de código, já é possível criar programas totalmente funcionais.

Por que usar o Google Colab?

SEU AMBIENTE NA NUVEM

O Google Colaboratory (Colab) é a ferramenta que vamos usar durante as aulas para escrever e executar nossos códigos em Python.

Gratuito

Basta ter uma conta Google para acessar todos os recursos básicos.

Sem instalação

Funciona direto no navegador, sem precisar instalar nada no computador.

Salva no Drive

Os notebooks ficam salvos automaticamente no seu Google Drive.

Fácil de compartilhar

Envie o link e qualquer pessoa pode ver ou editar o código.

Texto e código

Mistura explicações em texto e blocos de código no mesmo documento.

Processamento

Acesso a processamento poderoso na nuvem sem depender do seu hardware.

ACESSE AGORA:

colab.research.google.com

Primeiro contato: print e variáveis

SEU PRIMEIRO PROGRAMA EM PYTHON

O comando print

Todo programador começa com o famoso "Olá, mundo!". Em Python, exibir uma mensagem na tela é simples assim:

PYTHON

```python
print("Olá, mundo!")
```

O comando **print()** é usado para mostrar informações na tela.

Variáveis

Variáveis armazenam informações para usar no programa. Pense nelas como "caixinhas" com um nome e um valor guardado dentro.

PYTHON

```python
nome = "Maria"
idade = 20
altura = 1.65

print(nome)    # Maria
print(idade)   # 20
print(altura)  # 1.65
```

Você pode criar quantas variáveis quiser, sempre usando nomes descritivos para facilitar a leitura do código.

Tipos de dados

PYTHON RECONHECE DIFERENTES TIPOS DE INFORMAÇÃO

Cada valor em Python tem um **tipo de dado.** Conhecer esses tipos é fundamental para manipular as informações corretamente no seu programa.

Use a função `type()` para descobrir o tipo de qualquer variável!

TIPO	DESCRIÇÃO	EXEMPLO
`str`	Texto (string)	"Carlos"
`int`	Número inteiro	18
`float`	Número decimal	8.5
`bool`	Verdadeiro ou Falso	True / False

```
nome = "Carlos" # str
idade = 18 # int
nota = 8.5 # float
aprovado = True # bool

print(type(nome)) # <class 'str'>
```

Operações e entrada de dados

CALCULANDO E INTERAGINDO

Python realiza operações matemáticas de forma intuitiva e permite receber dados do usuário facilmente.

⚠ ATENÇÃO

O comando `input()` sempre retorna texto (string). Use `int()` ou `float()` para converter para número antes de fazer cálculos!

Operações Matemáticas

```
a = 10
b = 5

print("Soma:", a + b)          # 15
print("Subtração:", a - b)     # 5
print("Multiplicação:", a * b) # 50
print("Divisão:", a / b)       # 2.0
```

Entrada de Dados (Input)

```
nome = input("Digite seu nome: ")
print("Olá,", nome)

numero1 = int(input("Digite o primeiro número: "))
numero2 = int(input("Digite o segundo número: "))
print("Resultado da soma:", numero1 + numero2)
```

Estruturas condicionais

TOMANDO DECISÕES

Os condicionais permitem que o programa tome decisões com base em condições, executando diferentes blocos de código.

`if`	”se” a condição for verdadeira
`elif`	”senão, se” (outra condição)
`else`	”caso contrário” (se tudo falhar)

EXEMPLO 1: IF / ELSE

```
idade = int(input("Digite sua idade: "))

if idade >= 18:
    print("Você é maior de idade.")
else:
    print("Você é menor de idade.")
```

EXEMPLO 2: IF / ELIF / ELSE

```
nota = float(input("Digite a nota do aluno: "))

if nota >= 7:
    print("Aprovado")
elif nota >= 5:
    print("Recuperação")
else:
    print("Reprovado")
```

Estruturas de repetição

REPETINDO AÇÕES COM FOR E WHILE

As estruturas de repetição executam um bloco de código várias vezes, automatizando tarefas repetitivas.

Estrutura	Quando usar
for	Número de repetições conhecido (ex: percorrer uma lista ou intervalo).
while	Repetir até uma condição ser satisfeita (ex: aguardar entrada válida).

O laço for

```
for i in range(1, 6):
    print("Número:", i)

# Imprime de 1 a 5
```

O laço while

```
contador = 1

while contador <= 5:
    print("Contador:", contador)
    contador += 1

# Imprime de 1 a 5
```

Listas, Tuplas e Dicionários

ARMAZENANDO MÚLTIPLOS VALORES

Listas

Coleção ordenada e **mutável** de valores. Usamos colchetes `[]`.

```
frutas = ["maçã", "banana",
"uva"]

# Acessando item (índice 0)
print(frutas[0])  # maçã

# Adicionando novo item
frutas.append("abacaxi")
print(frutas)
```

Tuplas

Coleção ordenada e **imutável** (não pode ser alterada). Usamos parênteses `( )`.

```
dias = ("seg", "ter", "qua")

# Acessando item
print(dias[2])  # qua

# ERRO: Não é possível alterar
# dias[0] = "domingo"
```

Dicionários

Armazenam informações em pares de **chave e valor**. Usamos chaves `{}`.

```
aluno = {
    "nome": "João",
    "idade": 19,
    "curso": "Python"
}

# Acessando pelo nome da chave
print(aluno["nome"])   # João
print(aluno["idade"])  # 19
```

Boas práticas e próximos passos

PROGRAMAR BEM VAI ALÉM DE FAZER FUNCIONAR